DNN Programming Assignment 2

RNN, LSTM, GRU and Transformer Experiments

Korada Sriya Kushal Kalyan **2026AJ05601** DNN

Scope. From-scratch recurrent and Transformer components were evaluated on UCI HAR, the synthetic adding problem, and Tiny Shakespeare. Exact per-run values are retained in `results.csv`.

1 Problem 1: recurrent models

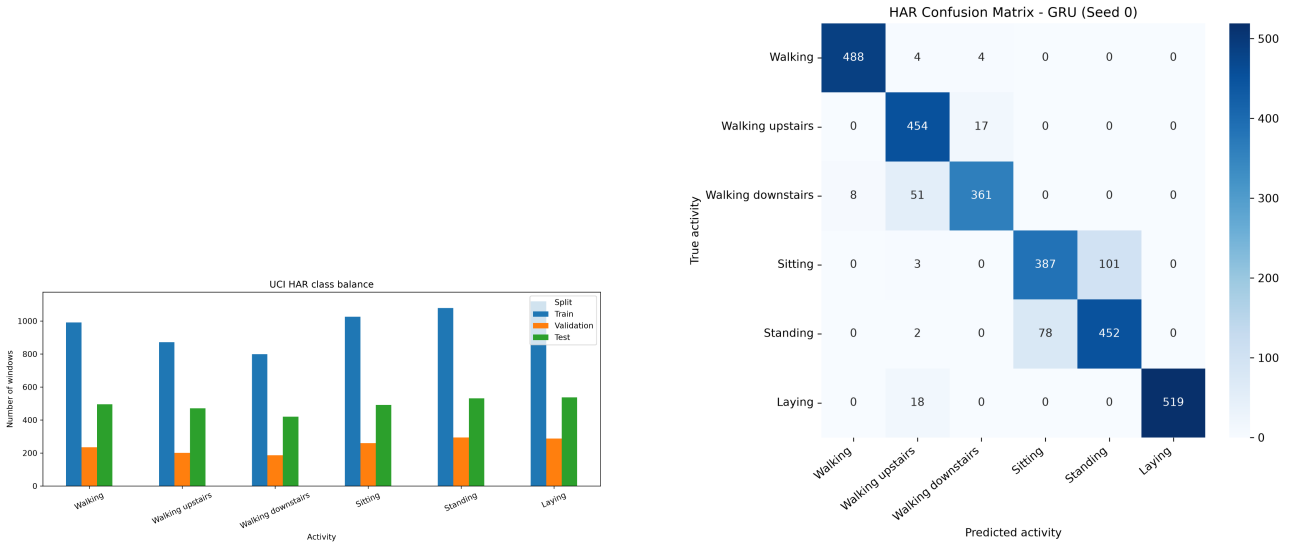
1.1 Implementation and HAR classification

Vanilla RNN, LSTM and GRU cells were implemented directly from their equations. For input size 9 and hidden size 64, custom-cell parameter counts were 4,736, 18,944 and 14,208. Copied-weight comparisons with PyTorch were within 10^{-5}. UCI HAR has 128 time steps, 9 channels and 6 classes. Subject separation was preserved and normalization statistics came only from the training subset. Models used hidden size 64, Adam (10^{-3}), batch size 64, 30 epochs and three seeds. The class split

Table 1: HAR recurrent-model results (mean ± std).

Model	Accuracy	Macro-F1	Params	s/epoch
RNN	0.7465±0.0427	0.7365±0.0499	5,190	3.53
LSTM	0.8837±0.0177	0.8806±0.0240	19,590	4.41
GRU	**0.8948±0.0073**	**0.8949±0.0070**	14,790	8.94

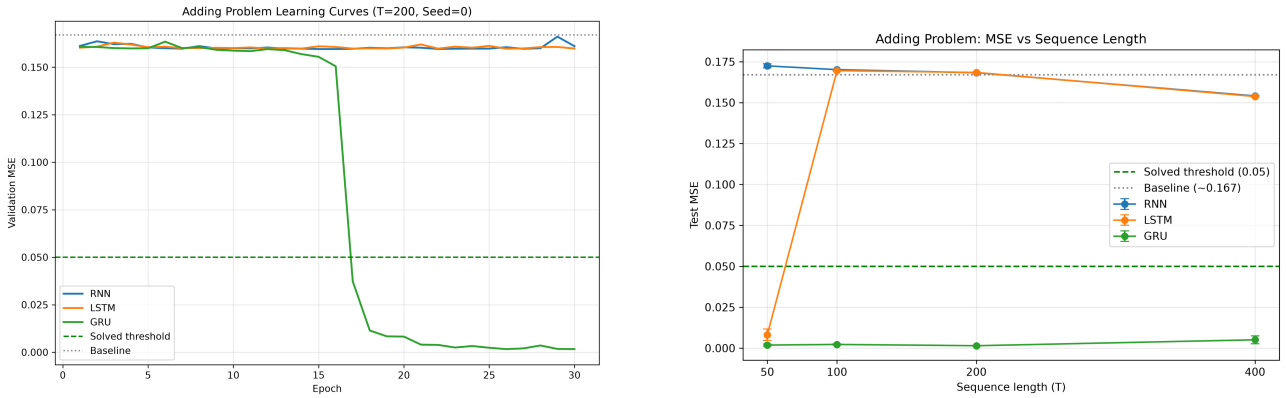
remained broadly balanced across all six activities (Figure 1). The best individual run was GRU seed 0 (90.30% accuracy; macro-F1 0.9025); Figure 2 shows that the principal errors are sitting versus standing and the two stair classes.



Figures 1–2. UCI HAR class balance (left) and GRU seed-0 confusion matrix (right).

1.2 Adding problem, gradients and capacity

A run was solved when test MSE < 0.05, against a constant baseline near 0.167. Figure 3 shows the required learning curves at $T = 200$: only GRU crosses the solved threshold. RNN solved no tested length, LSTM solved all seeds only at $T = 50$, and GRU solved all seeds at $T = 50, 100, 200, 400$.



Figures 3–4. Adding-problem learning curves at $T = 200$ (left) and test MSE versus sequence length (right).

At $T = 100$, post-training early-timestep gradient norms were 2.44×10^{-5} (RNN), 3.34×10^{-4} (GRU) and 5.76×10^{-10} (LSTM).

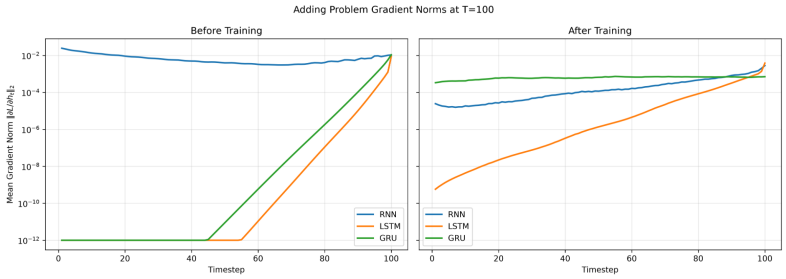


Figure 5. Hidden-state gradient norms before and after training at $T = 100$.

The hidden-size sweep shows diminishing returns: GRU accuracy increased from 0.8804 at size 16 (1,398 parameters) to 0.8988 at size 128 (54,150 parameters); LSTM increased from 0.8171 (1,830) to 0.8924 (71,942).

One-Page Analysis

Gradient Behaviour and Long-Term Memory

The gradient analysis at $T = 100$ shows clear differences in how the three recurrent architectures propagate information through time. Before training, the RNN gradient norm at the earliest timestep was 2.45×10^{-2}, compared with 1.40×10^{-20} for GRU and 9.70×10^{-23} for LSTM. After training, the corresponding early-timestep gradient norms were 2.44×10^{-5}, 3.34×10^{-4} and 5.76×10^{-10}. At the final timestep, post-training norms were 2.81×10^{-3} for RNN, 7.07×10^{-4} for GRU and 3.83×10^{-3} for LSTM. Figure 5 shows that propagation depends on both architecture and sequence position. The trained GRU retains a substantially larger early-sequence gradient, evidence for preserving information across longer distances.

Effect of Increasing Sequence Length

RNN failed at every tested sequence length. Mean MSE changed from 0.172421 ± 0.001270 at $T = 50$ to 0.154049 ± 0.000205 at $T = 400$, with 0/3 solved runs. LSTM achieved 0.008159 ± 0.003505 at $T = 50$ with 3/3 solved runs, but at $T = 100$ MSE rose to 0.169495 ± 0.000364 and it solved 0/3 runs at $T = 100, 200, 400$. GRU mean MSE was 0.001885, 0.002276, 0.001480 and 0.005106 for $T = 50, 100, 200, 400$, with 3/3 solved runs throughout. Figure 2 therefore identifies GRU as the most reliable long-range model here.

Accuracy, Parameters, and Training-Time Trade-off

GRU achieved 0.8948 ± 0.0073 HAR accuracy and 0.8949 ± 0.0070 macro-F1 using 14,790 parameters. LSTM achieved 0.8837 ± 0.0177 and 0.8806 ± 0.0240 with 19,590 parameters, while RNN achieved 0.7465 ± 0.0427 and 0.7365 ± 0.0499 with 5,190. GRU increased from 0.8804 accuracy and 1,398 parameters at hidden size 16 to 0.8988 and 54,150 at size 128; time rose from 6.24 to 17.36 seconds/epoch. LSTM rose from 0.8171 with 1,830 parameters to 0.8924 with 71,942. Larger models therefore produced diminishing accuracy gains at considerable cost.

Deployment Choice for a Battery-Powered Wearable

I would select **GRU-32**. It achieved 0.8921 accuracy and 0.8920 macro-F1 with 4,326 parameters and about 7.40 seconds/epoch. GRU-128 reached 0.8988 but required 54,150 parameters and 17.36 seconds/epoch. GRU-32 sacrifices only 0.68 percentage points while using about 92% fewer parameters. It also matched LSTM-128 accuracy (0.8921 versus 0.8924) with 4,326 rather than 71,942 parameters.

Individual Ablation Observation

At $T = 200$, LSTM forget bias 0 produced 0.168581 ± 0.000256 MSE and bias 1 produced 0.168532 ± 0.000084. Neither solved any seed. Bias initialization alone was insufficient to resolve the long-memory difficulty.

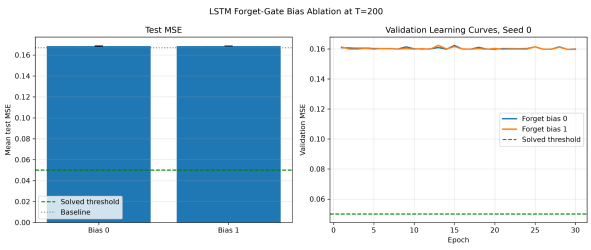


Figure 9. LSTM forget-gate-bias ablation at $T = 200$.

2 Problem 2: Transformer from scratch

2.1 Attention and encoder verification

Scaled attention $\text{softmax}(QK^\top/\sqrt{d_k})V$ passed row-sum, PyTorch agreement ($< 10^{-5}$), and causal-mask tests. The three-token manual output $[0.8022, 0.5989;\ 0.5989, 0.8022;\ 0.7518, 0.7518]$ agreed with code within 10^{-3}. Multi-head attention and a pre-LayerNorm residual encoder used FFN width $4d$, dropout and sinusoidal positions. At $d = 128, d_{ff} = 512$, the derived and coded encoder-block count was 198,272.

2.2 HAR Transformer

Each HAR timestep was projected to $d = 128$, a CLS token was prepended, and two encoder layers classified CLS. Learned positions added $129 \times 128 = 16,512$ parameters. Compared with GRU mean

Table 2: HAR positional ablation.

Position	Test acc.	Params	s/epoch
None	90.26%	398,726	1.65
Sinusoidal	89.38%	398,726	1.66
Learned	**90.94%**	415,238	1.69

accuracy 89.48%, the Transformer improved by 1.46 points but used about 28.1× more parameters.

2.3 Tiny Shakespeare causal language model

The 65-symbol model used context 128, $d = 128$, four heads, four blocks, $d_{ff} = 512$, dropout 0.1 and 809,793 parameters. At 2,000 steps, validation loss was **1.7841 nats**, perplexity 5.95 and BPC $= 1.7841/\ln 2 =$ **2.5739**.

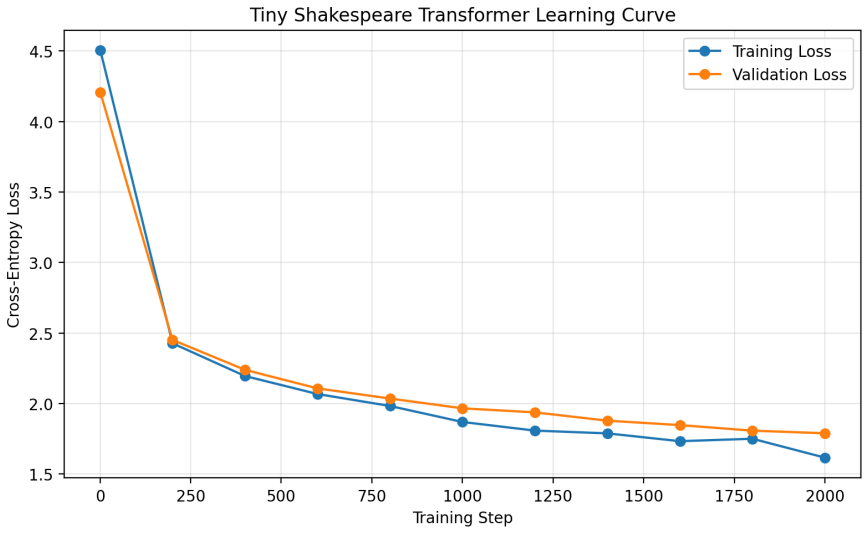


Figure 8. Tiny Shakespeare training and validation loss.

Top-k generation ($k = 20$) showed conservative repetition at temperature 0.2, balanced structure/diversity at 0.8, and more malformed language at 1.5. Full samples are in `generated_samples.txt`.
Overall conclusion. GRU was strongest on the long-memory recurrent tasks, GRU-32 offered the best deployment balance, learned-position attention improved HAR accuracy at higher parameter cost, and causal masking was essential for honest autoregressive generation.

2.4 Attention and head-count ablations

Both final-layer heads have a strictly dark upper triangle, proving zero future attention, but their lower-triangular patterns differ.

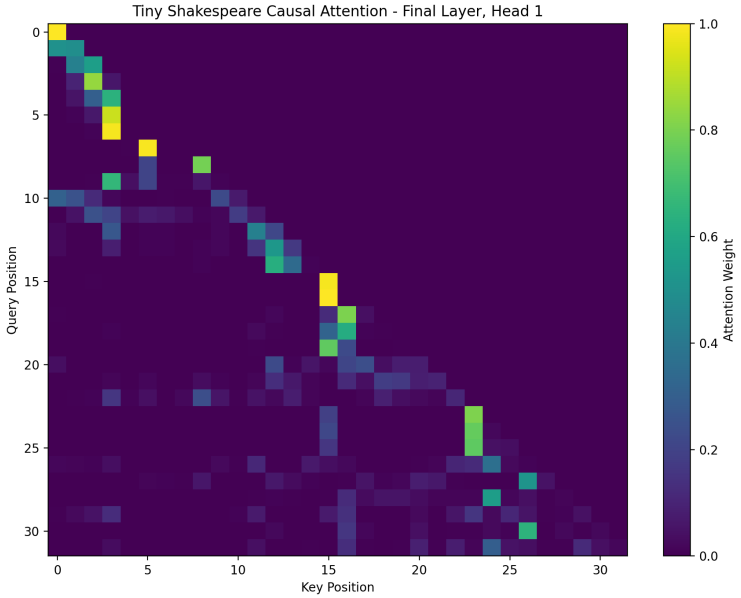


Figure 9. Final-layer causal attention, head 1.

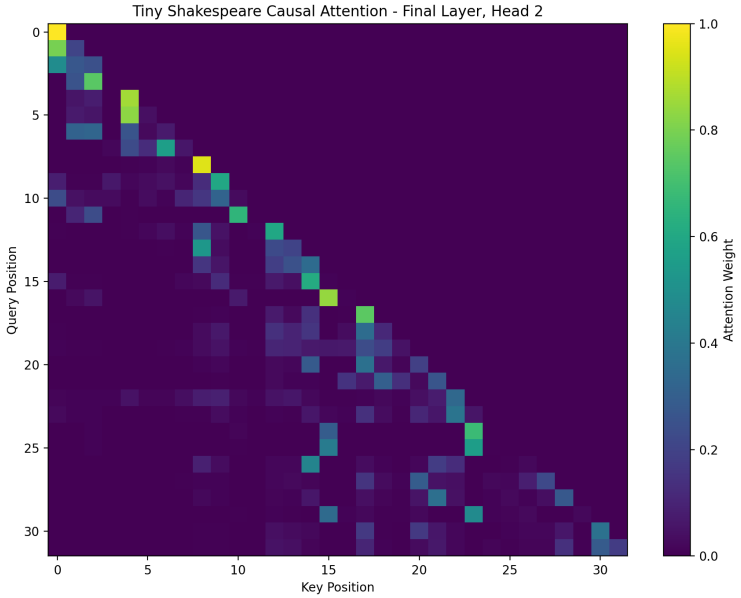


Figure 10. Final-layer causal attention, head 2, same input.

At fixed $d = 128$, all 1/4/8-head models had 809,793 parameters. Four heads performed best (loss 1.7841, perplexity 5.95), one head was close (1.7876, 5.98), and eight heads degraded (1.8086, 6.10).

2.5 Causal mask and context scaling

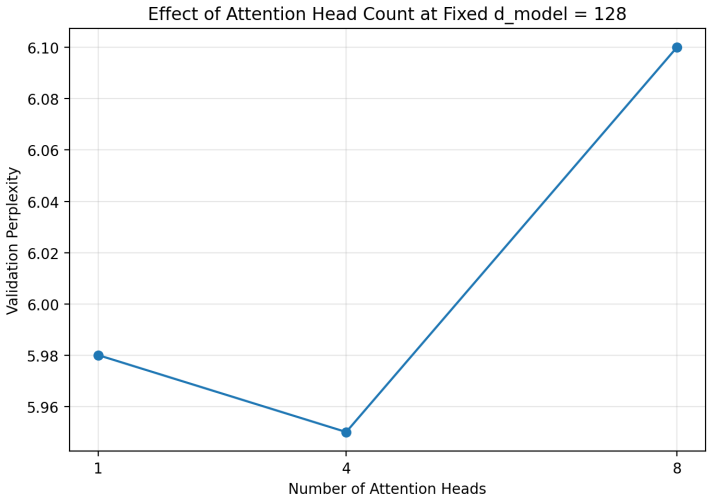


Figure 11. Validation perplexity versus attention head count.

Without a causal mask, validation loss fell misleadingly to 0.0191 (perplexity 1.02) because of future-token leakage; autoregressive generation then degenerated.

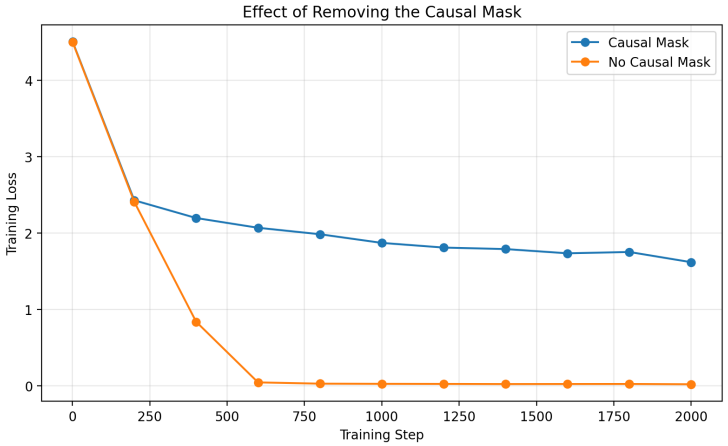


Figure 12. Training loss with and without the causal mask.

Contexts 64, 128, 256 and 512 required 13.12, 14.47, 13.26 and 29.74 ms/step, with 79.67, 101.36, 163.61 and 377.62 MB peak memory. The attention term is $O(n^2 d)$; fixed projections, FFNs and GPU parallelism explain why end-to-end growth is not exactly quadratic at shorter lengths.

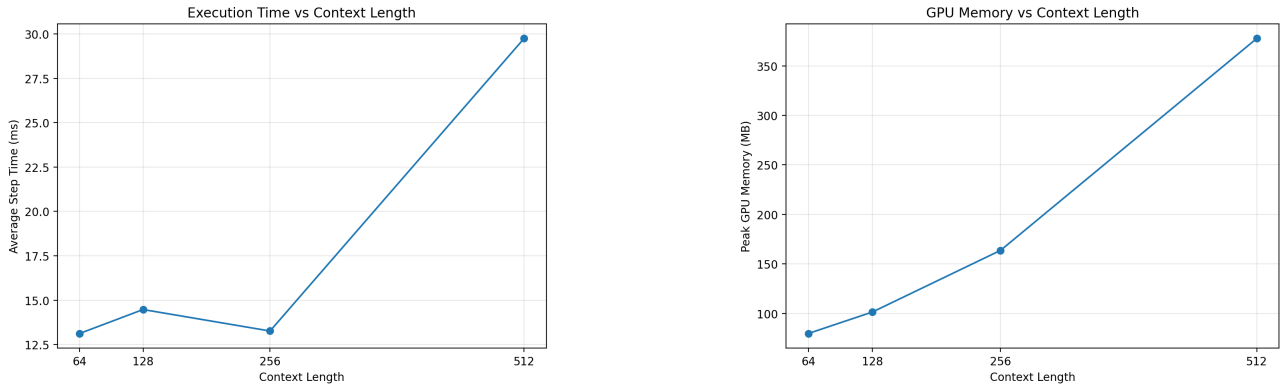


Figure 13. Execution time and peak GPU memory versus context length.

AI-use statement (excluded from six-page main report)

Microsoft Copilot was used for conceptual clarification, debugging assistance, pseudocode, notebook organization, Markdown formatting, result interpretation and report preparation. The from-scratch implementation, execution, experiments and outputs were completed and verified by the student.

References (excluded from six-page main report)

[1] A. Vaswani et al., "Attention Is All You Need," NeurIPS, 2017.
[2] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, 1997.
[3] K. Cho et al., "Learning Phrase Representations using RNN Encoder–Decoder," EMNLP, 2014.
[4] D. Anguita et al., "A Public Domain Dataset for Human Activity Recognition Using Smartphones," ESANN, 2013.
[5] A. Karpathy, char-rnn / Tiny Shakespeare corpus, 2015.
[6] PyTorch Contributors, PyTorch Documentation.